Aluno: Dérick Willer Borges Monteiro
Matrícula: 202200503

O programa

1. **Como executar o programa?**
   É só abrir o index.html no navegador. Continuei em cima do programa da atividade 05, então segue em HTML, CSS e JavaScript, sem biblioteca.
2. **Como usar?**
   No menu Primitivas, em Polígonos, escolhe Varredura ou BoundaryFill. Abre uma janela pedindo a quantidade de faces (de 3 a 50). Depois é só clicar na tela essa quantidade de vezes, cada clique é um vértice. Enquanto vai clicando aparece uma prévia das arestas, e no último clique o polígono é desenhado e já sai preenchido. O contorno fica na cor do algoritmo e o preenchimento num tom mais escuro. A quantidade de faces fica guardada para os próximos polígonos, e dá para trocar no botão "Alterar quantidade de faces" do painel. O Esc cancela um polígono pela metade.
3. **Como o contorno é desenhado?**
   Cada aresta liga um vértice ao próximo, e a última liga o último vértice de volta ao primeiro. Para cada aresta usei o Bresenham de reta da atividade 05.

a) Varredura

1. **Como o algoritmo funciona?**
   Ele percorre o polígono linha por linha, do menor y até o maior. Em cada linha acha os x onde ela cruza as arestas, ordena e pinta entre os pares: do 1º ao 2º, do 3º ao 4º e assim por diante. Dentro de um par está dentro do polígono, entre um par e outro está fora.
2. **Como são guardadas as arestas?**
   Monto uma tabela de arestas separada pelo y de baixo de cada uma. Cada aresta guarda o y de cima, o x no y de baixo e quanto o x anda quando o y sobe 1, que é 1/m = dx/dy. Aresta horizontal fica de fora porque não cruza nenhuma linha. Quando a varredura chega no y de baixo a aresta entra na lista de arestas ativas, e quando chega no y de cima ela sai. Assim cada linha só olha as arestas ativas, e o x novo é só somar 1/m no anterior, sem calcular a interseção de novo.
3. **E quando a linha passa bem em cima de um vértice?**
   O vértice é o fim de uma aresta e o começo de outra, então ele contaria duas vezes e bagunçaria os pares. Para não acontecer isso a aresta vale do y de baixo até um antes do y de cima. O pixel que fica faltando nessa linha o contorno cobre.

b) BoundaryFill

1. **Como o algoritmo funciona?**
   Primeiro a borda do polígono é desenhada. Depois ele começa num pixel de dentro, a semente, pinta esse pixel e olha os vizinhos. Todo vizinho que não é borda e ainda não foi pintado também é pintado, e assim vai espalhando até bater na borda por todos os lados.

2. **Por que 4 vizinhos e não 8?**
   A borda feita com Bresenham anda na diagonal em vários pontos, então ela só fecha para quem anda para cima, baixo, esquerda e direita. Com 8 vizinhos o preenchimento passaria entre dois pixels da borda que só se encostam pela quina e vazaria para fora.
3. **Como é escolhida a semente?**
   O enunciado pede que o polígono saia preenchido sozinho, então o programa procura a semente. Ele percorre a área do polígono e pega um pixel que não é borda e que está dentro pela regra par-ímpar (uma linha saindo do ponto para a direita cruza um número ímpar de arestas). Polígono côncavo pode ficar com mais de uma região fechada, então isso se repete até não sobrar pixel de dentro sem pintar. Também deixei uma margem de 1 pixel em volta do polígono, e se o preenchimento chegar nela é porque vazou, aí essa região é descartada.
4. **Por que usar uma pilha em vez de recursão?**
   Na versão recursiva cada pixel chama a função de novo, e num polígono grande isso estoura a pilha do navegador. Então guardo os pixels que faltam visitar numa pilha e vou tirando um de cada vez, o resultado é o mesmo.

Comparação

1. **Os dois algoritmos dão o mesmo resultado?**
   Nos testes que fiz sim, pintaram exatamente os mesmos pixels, inclusive em polígono côncavo e em estrela. Na estrela o miolo fica vazio nos dois, porque os dois seguem a regra par-ímpar.
2. **Qual é a diferença entre eles?**
   A varredura trabalha direto com as arestas, linha por linha, e não precisa da borda desenhada. O BoundaryFill depende da borda já estar pintada e vai de pixel em pixel olhando os vizinhos, então gasta mais memória e passa mais vezes em cada pixel. Em compensação ele preenche qualquer região fechada, não só polígono.

Tela do programa



Figura 1 - Polígonos preenchidos com Varredura (rosa) e BoundaryFill (amarelo)